FH AACHEN
UNIVERSITY OF APPLIED SCIENCES

FH Aachen

Bachelor's Thesis

in partial fulfillment of the requirements for the degree of

B. Sc. in Computer Science

AI Agents in Software Development: Best Practices for Structured Human-Agent Collaboration

Fakih Lanjri Houdaifa

Matriculation Number 3574047 Aachen,

First Examiner:

Second Examiner:

Declaration

I hereby declare that I have written this thesis independently and have not used any sources other than those indicated in the bibliography. Passages that have been taken verbatim or in substance from published or as yet unpublished sources are identified as such. The drawings or illustrations in this thesis were created by myself or are provided with an appropriate reference to their source. This thesis has not been submitted in the same or similar form to any other examination authority.

Aachen,

Contents

1 Introduction

1.1 Background

Large Language Models have moved a long way from being simple autocomplete engines for code. Today, tools built on top of them act as genuine coding agents: they can read a set of requirements, plan an implementation, write entire features, generate their own tests, and reason, at least to some degree, about the architecture of the software they are building. Modern coding agents can be integrated directly into conventional development environments, allowing developers to hand off substantial parts of the implementation work to an AI agent while still reviewing and supervising what it produces.

More recently, agent-first development environments have emerged that extend this concept further. Rather than relying on a single agent working through a linear sequence of tasks, these environments can support the orchestration of several specialized agents, each responsible for different parts of the development workflow, from planning and architectural design to implementation and verification.

This shift changes the nature of software development in a subtle but important way. In traditional development, the person writing the code is also the one reasoning about it. When an AI agent takes over the implementation, a layer is inserted between the idea and the code: the agent interprets instructions, makes its own architectural choices, and produces artifacts that a human then has to review, correct, or approve. How good the resulting software turns out to be depends not only on how capable the underlying model is, but just as much on how the collaboration between human and agent is set up in the first place — how precise the prompts are, whether any architectural boundaries are defined at all, and what kind of review process, if any, the agent's work has to pass.

1.2 Motivation

This thesis grew out of a practical project in which software was built from the ground up using AI coding agents, across two different development environments that each embody a different philosophy of working with an agent.

The first part of the project was carried out with Claude Code inside Visual Studio Code. Development here followed a deliberately loose approach: the agent was asked to implement features from scratch with very little structural guidance, beyond a general description of the software idea, the functional and non-functional requirements, and the core modules of the web application. The goal was to observe how an AI agent behaves when given substantial implementation freedom within a single-agent, conversational workflow — and where exactly that freedom starts to cause problems, whether in the form of inconsistency, architectural drift, or small but consequential implementation mistakes.

What came out of that first part shaped the second one directly. For the second part, development moved to Google Antigravity, an agent-first development environment organized around orchestrating multiple agents rather than interacting with a single one. Instead of assuming that better software simply requires a more capable model, this second phase focused on something different: the architectural rules, review steps, and development discipline imposed on the agents themselves. A more structured methodology was introduced for this purpose, built around an AI Development Charter, a three-agent setup (an Architect, an Implementer, and a Post-Implementation Architecture Review Agent), mandatory design reviews before any code was written, and a formal testing strategy — all of it designed to make deliberate use of the environment's support for parallel, role-specialized agents working together, rather than one agent working through a single, linear thread of tasks.

The motivation behind this thesis is therefore twofold. First, it aims to document and compare, in concrete terms, what changes when moving from an unstructured, single-agent approach to a structured, multi-agent approach in AI-assisted software development. Second, it aims to distill from that comparison a set of best practices for human-agent collaboration that hold up beyond the specific tools used here, and that can inform how developers approach this kind of work more generally.

1.3 Problem Statement

AI coding agents are being increasingly adopted in both industry and academic settings. However, systematic understanding of how the structure of human-agent collaboration — rather than solely the capability of the underlying model — influences the quality, architectural soundness, and reliability of resulting software remains limited. Existing research on AI-assisted software development has often focused on model capabilities, benchmarks, or individual code-generation tasks, while broader aspects of the development process —

including requirements interpretation, architectural planning, iterative implementation, testing, review, and documentation maintenance — require further investigation.

Several specific challenges motivate the work presented in this thesis:

1. **Limited structural comparison.** Existing work provides limited empirical comparison between unstructured, prompt-driven interaction with AI coding agents and more structured, architecture-first development approaches, particularly when both approaches are examined within the same software project.

2. **Unclear impact of architectural constraints.** It remains unclear to what extent architectural rules, mandatory design reviews, and the separation of responsibilities across specialized agents can improve the quality and consistency of AI-generated software compared with a single agent operating with fewer structural constraints.

3. **Gaps in testing and validation.** AI coding agents can generate tests that execute successfully without necessarily providing sufficient validation of the intended behavior. This raises the question of how testing and verification should be structured when substantial parts of both implementation and test generation are delegated to AI agents.

4. **Documentation drift.** As AI agents modify software iteratively, maintaining consistency between the implementation and related specifications, such as OpenAPI documents, becomes an important maintainability challenge.

5. **Limited consolidated guidance.** Despite the growing adoption of AI coding agents, empirically grounded guidance on how developers should structure human-agent collaboration across prompting, architecture, review, testing, and documentation remains fragmented.

1.4 Objectives of the Thesis

The aim of this thesis is to systematically document, analyze, and compare two contrasting approaches to AI-agent-assisted software development — one unstructured and one structured — based on a practical project, and to derive from this comparison concrete, actionable best practices for human-agent collaboration in software engineering.

More specifically, this thesis sets out to:

1. **Document the unstructured approach (Part 1).** Describe how the software was developed using a single-agent workflow with minimal architectural guidance, and analyze the resulting architectural limitations, the influence of prompt precision, and the extent to which human supervision was required.

2. **Document the structured approach (Part 2).** Describe the design and implementation of a structured, multi-agent development methodology, including the AI Development Charter, parallel role-specialized agents, mandatory design reviews, the three-tier testing strategy, and the automation of OpenAPI specifications to prevent documentation drift.

3. **Evaluate each approach.** Define evaluation criteria and assess each development approach with respect to code quality, test validity, architectural consistency, documentation consistency, and development efficiency.

4. **Compare the two approaches.** Conduct a direct cross-approach comparison to determine how the structured methodology affected the limitations observed during the unstructured development phase, including differences in feature implementation, code and test quality, architectural consistency, and documentation reliability.

5. **Derive best practices.** Bring the findings together into a set of best practices for human-agent collaboration, covering prompt precision, the role of human oversight, architectural guardrails, recommendations for multi-agent workflows, and strategies for preventing documentation drift.

Taken together, these objectives position the thesis as both an empirical case study of AI-agent-assisted software development and a practically oriented source of recommendations that developers and organizations can draw on when structuring their own collaboration with AI coding agents.

2 Foundations

3 Methodology

3.1 Structure of the Practical Project (Two Sub-Projects)

3.2 Tools and Models Used

3.3 Evaluation Criteria

4 Part 1: Unstructured Development

4.1 Initial Situation and Objectives

4.2 Prompting Without a Structural Approach

4.3 System Prompt Context

4.4 Development of the First Feature (Login/Registration)

4.5 Test Suites and Test Case Generation

4.6 Architectural Analysis of the Generated Software

4.7 Observations: Prompt Precision, Human Oversight, Architectural Limitations

4.8 Critical Problems and the Test Failure Analysis Principle"

5 Part 2: Structured Approach (AntiGravity)

5.1 Motivation for a Structured Approach

5.2 AI Development Charter and Architectural Rules

5.3 Parallel Agent Architecture (Architect / Implementer / Post-Implementation Review Agent)

5.4 Design Reviews Before Coding (implementation_plan.md)

5.5 Testing Strategy: Force Requirement Validation & Three-Tier Testing

5.6 Project Knowledge Files

5.7 OpenAPI Specification Automation and Avoiding Documentation Drift

5.8 Identification of Architectural Problems and Weaknesses

6 Comparison of Both Approaches

6.1 Comparison: Part 1 vs. Part 2

6.2 Feature Diversity

7 Evaluation

7.1 Evaluation Criteria and Methodology

7.2 Quality of Generated Code

7.3 Quality and Validity of Generated Tests

7.4 Architectural Conformance and Documentation Consistency

7.5 Efficiency and Effort (Human vs. Agent)

7.6 Summary Assessment: Part 1 vs. Part 2

8 Best Practices for Human-Agent Collaboration

8.1 Importance of Prompt Precision and Context Design

8.2 Role of Human Oversight (Human-in-the-Loop)

8.3 Architectural Guardrails for AI Agents

8.4 Recommendations for Multi-Agent Workflows

8.5 Avoiding Documentation Drift

9 Discussion

9.1 Opportunities and Limitations of AI Agents in Software Development

9.2 Implications for Practice

10 Conclusion and Outlook

Bibliography